

```
In [25]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

```
In [26]: df = pd.read_csv("creditcard.csv")

X = df.drop(['Class'], axis=1)
y = df['Class']
```

```
In [27]: scaler = StandardScaler()
X[['Time', 'Amount']] = scaler.fit_transform(X[['Time', 'Amount']])
```

```
In [28]: imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)
```

```
In [29]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
In [30]: model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred_before = model.predict(X_test)
```

```
In [31]: print("✅ BEFORE SMOTE")
print(classification_report(y_test, y_pred_before))
```

```
✅ BEFORE SMOTE
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.86	0.60	0.71	98
accuracy			1.00	56962
macro avg	0.93	0.80	0.85	56962
weighted avg	1.00	1.00	1.00	56962

```
In [32]: smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
```

```
In [33]: model_sm = LogisticRegression(max_iter=1000)
model_sm.fit(X_train_sm, y_train_sm)
y_pred_after = model_sm.predict(X_test)
```

```
In [34]: print("✅ AFTER SMOTE")
print(classification_report(y_test, y_pred_after))
```

✓ AFTER SMOTE

	precision	recall	f1-score	support
0	1.00	0.97	0.99	56864
1	0.06	0.93	0.11	98
accuracy			0.97	56962
macro avg	0.53	0.95	0.55	56962
weighted avg	1.00	0.97	0.99	56962

```
In [35]: import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
import numpy as np

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 1. Data shape
print("x_train shape:", x_train.shape)
print("y_train shape:", y_train.shape)

# 2. Show a few images
for i in range(3):
    plt.imshow(x_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.show()

# 3. Count of each digit
unique, counts = np.unique(y_train, return_counts=True)
print("Digit counts:", dict(zip(unique, counts)))

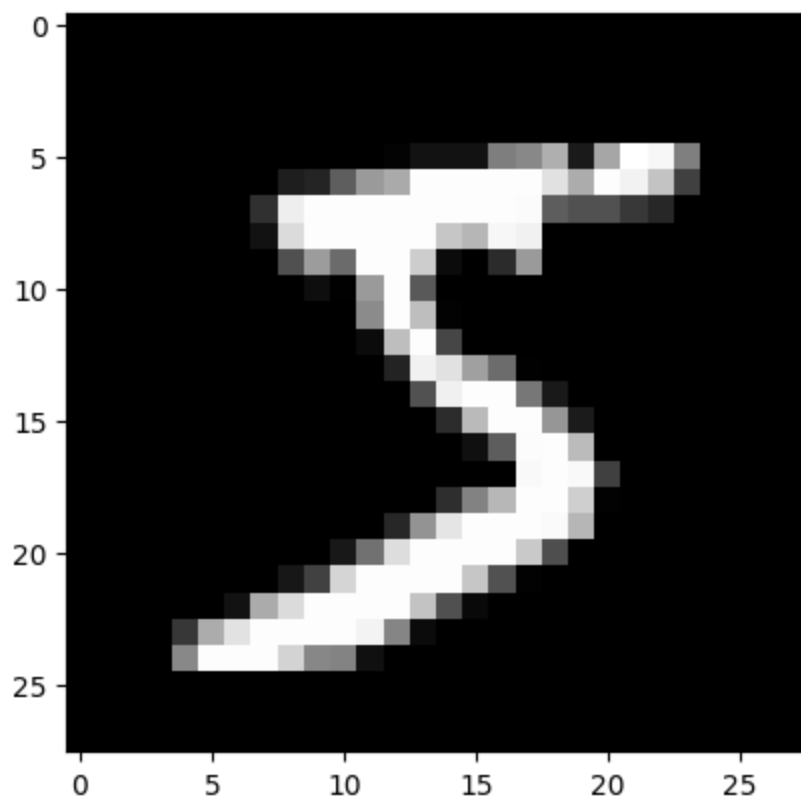
# 4. Pixel value range
print("Min pixel:", x_train.min(), "Max pixel:", x_train.max())

# 5. Average image of digit '0'
avg_zero = np.mean(x_train[y_train == 0], axis=0)
plt.imshow(avg_zero, cmap='gray')
plt.title("Average image of digit 0")
plt.show()
```

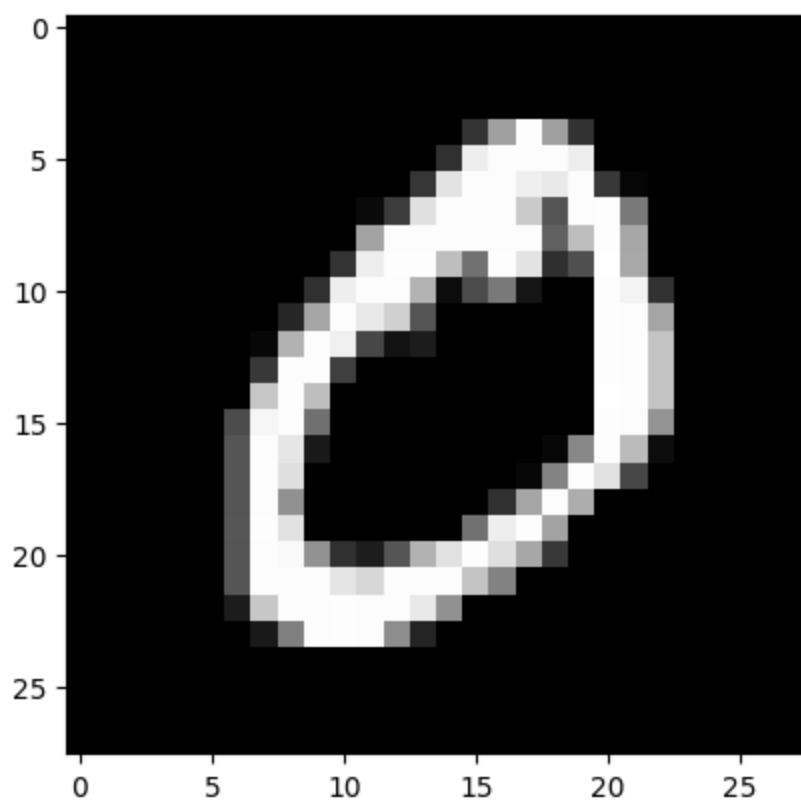
x_train shape: (60000, 28, 28)

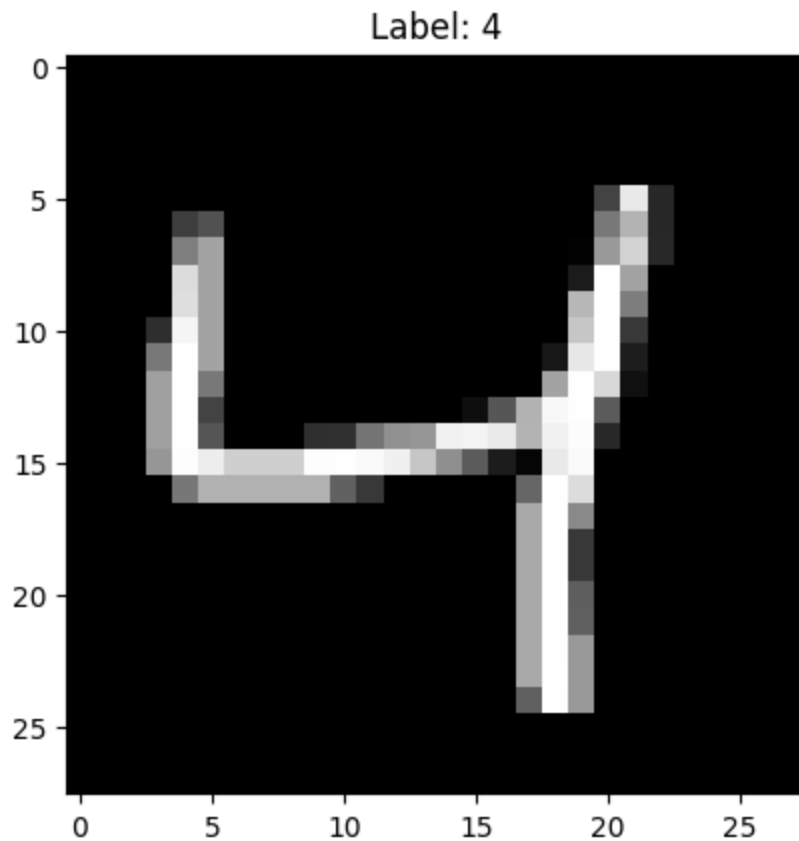
y_train shape: (60000,)

Label: 5



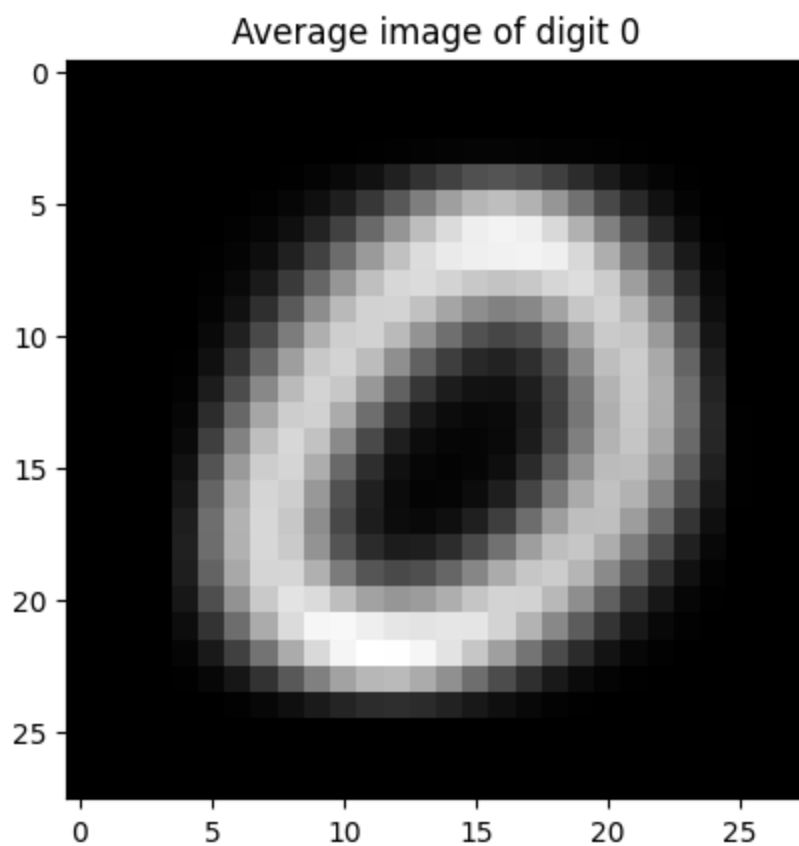
Label: 0





Digit counts: {np.uint8(0): np.int64(5923), np.uint8(1): np.int64(6742), np.uint8(2): np.int64(5958), np.uint8(3): np.int64(6131), np.uint8(4): np.int64(5842), np.uint8(5): np.int64(5421), np.uint8(6): np.int64(5918), np.uint8(7): np.int64(6265), np.uint8(8): np.int64(5851), np.uint8(9): np.int64(5949)}

Min pixel: 0 Max pixel: 255



In []: